The 3 Control Planes

The 3 Control Planes

1) Internal Control Plane (ICP)

Lives: inside the AXION system runtime
Primary user: the **internal agent** (kit/spec generator) + operators
Purpose: run AXION's pipeline deterministically and enforce system contracts.

Core responsibilities

- Run orchestration (run state machine, stage sequencing, reruns/resume)
- Registry loading + pin resolution (standards/templates/gates/schema versions)
- Standards applicability + resolution → standards snapshot
- Template selection + rendering → render envelopes
- Gate evaluation + evidence requirements
- Proof ledger writing + run reports (stage reports, gate reports)
- Kit packaging (manifest, entrypoint, bundle export)
- Operator actions + audit/trace (override handling, approvals if enabled)

Hard boundaries

- ICP governs **AXION**, not the target app repo (except via produced kits/artifacts).
- ICP decides *what must be true* and produces artifacts that prove it.

---

2) External Control Plane (Kit Control Plane) (ECP / KCP)

Lives: inside the **Agent Kit** (ships with the kit)
Primary user: the **build agent** executing the kit
Purpose: enforce kit-local rules while the build is happening and keep "work + proof" aligned to AXION contracts.

Core responsibilities

- Kit validation (required artifacts present, schema checks, entrypoint compliance)
- Work unit guidance (read PLAN_UNITs, enforce 1-target rule, "next unit" helpers)
- Local verification runner (lint/test/build/typecheck) + log capture
- Result ingestion helpers (write RESULT files, attach implementation/proof refs)
- Proof capture and normalization (proof objects consistent with AXION proof types)
- Guardrails for execution (no forbidden paths, no copying rules, restricted content handling)
- Produce a kit-local run report that can be merged back into repo artifacts

Hard boundaries

- KCP does **not** change AXION registries or standards; it executes within the kit.
- KCP optimizes “build correctness + evidence,” not “system design.”

---

3) Maintenance Control Plane (MCP)

Lives: inside the **target repo**, outside AXION (repo-resident assistant)
Primary user: operators/devs + maintenance agent(s)
Purpose: keep the repo healthy over time—upgrades, migrations, test hardening—while staying compatible with AXION’s contracts.

Core responsibilities

- Dependency upgrades (version bumps, lockfile discipline, changelog notes)
- Migration management (schema migrations, backcompat checks, data repair scripts)
- Test suite evolution (add regressions, reduce flakiness, coverage improvements)
- Refactor support (safe restructuring, module boundaries, lint/type fixes)
- CI workflow upkeep (pipelines, caching policies, toolchain pins)
- AXION integration upkeep (ensure manifests, proofs, gates, and kit outputs still validate)
- Maintenance-mode execution (trigger MM-xx runs, enforce post-incident backfill)

Hard boundaries

- MCP does **not** redefine AXION system rules; it operates under them.
- MCP focuses on **post-build lifecycle**: evolution, hygiene, and long-term stability.

Quick mental model

- **ICP** = AXION’s “brain” (builds kits + enforces system contracts)
- **KCP** = Kit’s “foreman” (keeps build agent aligned + captures proof)
- **MCP** = Repo’s “maintenance supervisor” (upgrades/tests/migrations without drift)

Internal Control Plane (ICP)

Internal Control Plane (ICP) — Detailed Spec

Role

Lives: inside AXION runtime
Primary user: internal agent + operators
Purpose: execute AXION deterministically, enforce contracts, and produce auditable artifacts (kits + proof) that downstream execution can rely on.

---

1) ICP Inputs (what it consumes)

ICP must be able to load these inputs deterministically:

1.1 Run request / run context

- `run_id` (or generated)
- `mode_id` (MM-xx) and/or `run_profile_id`
- `risk_class` (prototype/production/compliance)
- `executor_type` default (internal/external for downstream kit)
- `targets` (platforms, stack choices, domains)

1.2 Project state (source-of-truth)

- registries/indexes (standards/templates/gates/proof types/schema registry/version registry)
- pinned versions or pin policy
- prior run snapshots (optional for diffs/repro)

1.3 Intake artifacts

- submission
- normalized input
- submission record

---

2) ICP State Model (run state machine)

ICP owns the canonical run lifecycle.

2.1 Run states (minimum)

- `QUEUED` → `RUNNING` → `GATED` → (`FAILED` | `RELEASED`) → `ARCHIVED`
- Optional: `PAUSED`, `CANCELLED`, `ROLLING_BACK`

2.2 Stage states (minimum)

- `NOT_STARTED`, `IN_PROGRESS`, `PASS`, `FAIL`, `SKIP`

2.3 Transition rules (deterministic)

- cannot advance stage unless:
  - prior stage `PASS` (or explicitly `SKIP` allowed by mode)
  - required artifacts exist and validate
- rerun rules:
  - stage rerun allowed only if it is idempotent or produces a new run sub-id
  - rerun must produce a new stage report and link to prior attempts

---

3) ICP Runtime Modules (what it actually does)

Each responsibility maps to a module that produces artifacts.

3.1 Run Orchestrator

Function: stage sequencing, reruns/resume, fail-fast, stage activation by mode/profile.

Produces

- `run_manifest`
- `stage_reports[]`
- `run_log` (trace)

Must enforce

- stage order contract (ORD)
- stage I/O contract (consumes/produces)

3.2 Registry Loader + Pin Resolver

Function: load registries, validate them, resolve pins and versions.

Produces

- `resolved_pinset/lockfile` for the run
- `registry_resolution_report` (optional but useful)

Must enforce

- schema validity for all registries

- referential integrity (IDs referenced exist)
- deterministic version selection (no implicit “latest”)

3.3 Standards Engine

Function: determine applicable standards and resolve conflicts into a snapshot.

Steps

1. compute applicability from run context
2. load packs + precedence rules
3. resolve into a single **standards snapshot**
4. pin snapshot version in run pinset

Produces

- `standards_applicability_output`
- `resolved_standards_snapshot`

Must enforce

- deterministic resolver order
- snapshot is complete (includes standard ids + versions + precedence decisions)

3.4 Template System Driver

Function: select templates and render them using canonical + standards snapshot.

Steps

1. select templates (deterministic)
2. render templates (placeholder/derive rules)
3. run completeness validation
4. package rendered outputs into envelopes

Produces

- `template_selection_result`
- `render_envelopes[]`
- `template_completeness_report`

Must enforce

- placeholder syntax rules
- completeness rules (no unresolved required placeholders)

3.5 Gate Engine (system-level enforcement)

Function: evaluate gates at defined gating points and produce objective results.

Inputs

- current artifacts produced so far
- evidence pointers (logs, files, hashes)
- configured gate set profile (from mode/profile)

Produces

- `gate_report` (global + per gate optional)

Must enforce

- predicate operator registry (no invented operators)
- evidence policy mapping (gate → required proof types)

3.6 Proof System (ledger + capture)

Function: capture run proofs and maintain the proof ledger.

Proof sources

- verification command outputs (logs + summaries)
- snapshot hashes
- selection outputs
- manual attestations (if allowed)

Produces

- `proof_objects[]`
- `proof_ledger` (append-only or revisioned)
- `verification_run_result`

Must enforce

- proof types must be registered
- proof pointers resolve
- redaction rules apply

3.7 Kit Packager

Function: package the kit deterministically for downstream execution.

Includes

- canonical spec package
- resolved standards snapshot
- rendered templates + envelopes
- plan/work units (if included by profile)
- verification commands policy
- kit control plane (if you ship it)
- manifest + entrypoint
- bundle export (zip/tar)

Produces

- `kit_manifest`
- `kit_entrypoint`
- `bundle_metadata`
- `bundle_export`

Must enforce

- kit completeness requirements (KIT)
- pinset is embedded
- reproducibility metadata present

3.8 Operator Actions + Audit/Trace

Function: record operator actions, overrides, approvals.

Actions

- start/stop/pause/resume run
- rerun stage
- approve override
- change pins (if permitted)
- release bundle

Produces

- `audit_log_entries[]`
- `override_records[]` (when used)

Must enforce

- never-overridable gate list
- override expiry + scope requirements
- all actions link to run_id

---

4) ICP Output Artifacts (the contract it guarantees)

The ICP must reliably output a **minimum artifact spine** every run:

Always produced

- `run_manifest`
- `run_log`
- `stage_reports`
- `gate_report` (even if failing)
- `pinset/lockfile`

Produced when relevant

- `standards_snapshot` (+ applicability output)
- `template_selection_result` + `render_envelopes`
- `proof_ledger` + proof objects + verification results
- `kit_manifest` + `entrypoint` + `bundle_export`
- `state_snapshot` / rollback pointer

---

5) Determinism guarantees

ICP must guarantee:

- same inputs + same pinset → same standards snapshot, template selection result, and rendered envelopes (modulo allowed noise fields)
- stage order does not vary
- resolver order does not vary
- outputs are written to deterministic paths

Allowed nondeterminism must be explicitly isolated (timestamps, run_id) and excluded from golden comparisons.

---

6) Failure semantics (what “fail” means)

When something fails:

- ICP must still produce:
    - gate report with remediation
    - logs + evidence pointers
    - stage report indicating where and why it stopped
- fail-fast if the failure blocks downstream correctness (schema invalid, missing artifacts, broken pins)

---

7) Hard boundaries (locked)

- ICP **does not** modify target app repos directly.
- ICP **does not** “invent” standards/templates/gates; it only uses registered items.
- ICP **does not** accept unverifiable success—PASS requires evidence per policy.
- ICP **must** write artifacts that downstream systems can validate without context.

ICP Input Contract

ICP Input Contract

ICP-IN-01 Run request / run context

Purpose: define *what kind of run this is* and what policy set applies.

Required fields

- `run_id` *(generated if not provided)*
- `mode_id` *(MM-xx)* **and/or** `run_profile_id`
- `risk_class` *(prototype | production | compliance)*
- `executor_type_default` *(internal | external) (default for kit execution constraints)*
- `targets`
  - `platforms` *(web | mobile | api | fullstack | other)*
  - `stack` *(optional: tech choices if known)*
  - `domains` *(feature domains relevant to applicability and template selection)*

Determinism rules

- if both `mode_id` and `run_profile_id` are provided, the precedence rule must be fixed (no guessing)
- `targets` must be normalized to a canonical form (ordering + casing + allowed enums)

ICP-IN-02 Project state (source-of-truth)

Purpose: load the system's authoritative rule/config surface.

Required inputs

- registries/indexes:
    - standards index (+ packs/applicability)
    - template registry (+ selection profiles)
    - gate index (+ gate set profiles + predicate operator registry)
    - proof type registry (+ evidence policy)
    - schema registry
    - version registry
    - terms/ID namespace registries (if used for validation)
- version control inputs:
    - `pinset/lockfile` *(if run is pinned)*
    - or `pin_policy` *(if run resolves pins at runtime)*

Optional inputs

- `prior_run_snapshots[]` *(for diff, repro, regression comparisons)*

Determinism rules

- no “latest” unless explicitly allowed by pin policy
- registry load order and merge order are fixed
- resolved pins must be emitted as a run artifact (pinset)

ICP-IN-03 Intake artifacts

Purpose: provide the factual input payload for canonical/spec generation.

Required artifacts

- `submission`
- `normalized_input`
- `submission_record`

Determinism rules

- `normalized_input` must be derived deterministically from `submission`
- `submission_record` must hash/reference the exact `submission` and `normalized_input`

ICP Input Validation Gates (minimum)

- **GATE-ICP-IN-01**: run context schema valid (mode/profile/risk/executor/targets)
- **GATE-ICP-IN-02**: all required registries present + schema-valid
- **GATE-ICP-IN-03**: pins resolved and emitted (pinset exists)
- **GATE-ICP-IN-04**: intake artifact chain valid (submission → normalized → record linked)

ICP State Model

ICP State Model (run state machine)

ICP owns the canonical run lifecycle.

2.1 Run states (minimum)

- **QUEUED**
- **RUNNING**
- **GATED**
- **FAILED**
- **RELEASED**
- **ARCHIVED**

Optional

- **PAUSED**
- **CANCELLED**
- **ROLLING_BACK**

2.2 Stage states (minimum)

- **NOT_STARTED**
- **IN_PROGRESS**
- **PASS**
- **FAIL**
- **SKIP**

2.3 Transition rules (deterministic)

Run-level transitions

- **QUEUED → RUNNING**
  - allowed when run context + required registries are loaded and validated
- **RUNNING → GATED**
  - occurs immediately after any configured gating point is reached
  - gate evaluation must run and produce a gate report
- **GATED → RUNNING**
  - allowed only if gating conditions are satisfied (gate report indicates pass for required gates)
  - if approvals/overrides are required, they must be recorded before transition
- **RUNNING → FAILED**
  - allowed when a non-recoverable failure occurs or a required gate fails with no override path
  - failure must record: stage_id, failure reason, and evidence pointers
- **RUNNING → RELEASED**
  - allowed only when all required stages are PASS (or permitted SKIP) and all required gates are PASS
  - release artifacts (kit/bundle + manifests) must exist and validate
- **FAILED → ARCHIVED**
  - allowed when run outputs are sealed for retention (reports, proofs, logs per policy)
- **RELEASED → ARCHIVED**

    - allowed when release outputs are sealed for retention and pinned

Optional transitions

- **RUNNING → PAUSED** (operator action)
- **PAUSED → RUNNING** (operator action)
- **RUNNING/PAUSED → CANCELLED** (operator action; must preserve audit trail)
- **FAILED/RELEASED → ROLLING_BACK** (if rollback is enabled)
- **ROLLING_BACK → ARCHIVED** (rollback outcome recorded)

Stage-level transitions

- **NOT_STARTED → IN_PROGRESS**
    - allowed only when stage activation conditions are met (mode/profile activates stage)
- **IN_PROGRESS → PASS**
    - allowed only when:
        - stage outputs exist
        - outputs validate against schemas
        - stage-level gates (if any) pass
- **IN_PROGRESS → FAIL**
    - allowed when:
        - required output missing, invalid, or a required verification step fails
    - must produce a stage report with remediation and evidence pointers
- **NOT_STARTED → SKIP**

    - allowed only when:
        - current mode/profile explicitly marks stage skippable
    - skip must be recorded in stage report with skip reason
- **FAIL** → **IN_PROGRESS** (rerun)
    - allowed only under rerun rules below

Advancement rule (no skipping the chain)

ICP cannot advance to stage *N+1* unless:

- stage *N* is **PASS**, or
- stage *N* is **SKIP** and SKIP is explicitly permitted by the active mode/profile, and
- all required artifacts for stage *N+1* inputs exist and validate

---

Rerun rules

- Stage rerun is allowed only if:
    - the stage is idempotent **or**
    - the rerun produces a new run sub-id / attempt id that isolates outputs
- A rerun must:
    - produce a new stage report
    - link to prior attempts (attempt chain)
    - preserve previous reports/logs (no overwrite without versioning)

ICP Runtime Modules

ICP Runtime Modules (what it actually does)

Each responsibility maps to a module that produces artifacts.

3.1 Run Orchestrator

Function: stage sequencing, reruns/resume, fail-fast, stage activation by mode/profile.

Produces

- `run_manifest`
- `stage_reports[]`
- `run_log` (trace)

Must enforce

- stage order contract (ORD)
- stage I/O contract (consumes/produces)

3.2 Registry Loader + Pin Resolver

Function: load registries, validate them, resolve pins and versions.

Produces

- `resolved_pinset/lockfile` for the run
- `registry_resolution_report` (optional)

Must enforce

- schema validity for all registries

- referential integrity (IDs referenced exist)
- deterministic version selection (no implicit “latest”)

3.3 Standards Engine

Function: determine applicable standards and resolve conflicts into a snapshot.

Steps

- compute applicability from run context
- load packs + precedence rules
- resolve into a single standards snapshot
- pin snapshot version in run pinset

Produces

- `standards_applicability_output`
- `resolved_standards_snapshot`

Must enforce

- deterministic resolver order
- snapshot completeness (standard ids + versions + precedence decisions)

3.4 Template System Driver

Function: select templates and render them using canonical + standards snapshot.

Steps

- select templates (deterministic)
- render templates (placeholder/derive rules)
- run completeness validation
- package rendered outputs into envelopes

Produces

- `template_selection_result`
- `render_envelopes[]`
- `template_completeness_report`

Must enforce

- placeholder syntax rules
- completeness rules (no unresolved required placeholders)

3.5 Gate Engine (system-level enforcement)

Function: evaluate gates at defined gating points and produce objective results.

Inputs

- current artifacts produced so far
- evidence pointers (logs, files, hashes)
- configured gate set profile (from mode/profile)

Produces

- `gate_report` (global; per-gate optional)

Must enforce

- predicate operator registry (no invented operators)
- evidence policy mapping (gate → required proof types)

3.6 Proof System (ledger + capture)

Function: capture run proofs and maintain the proof ledger.

Proof sources

- verification command outputs (logs + summaries)
- snapshot hashes
- selection outputs
- manual attestations (if allowed)

Produces

- `proof_objects[]`
- `proof_ledger` (append-only or revisioned)
- `verification_run_result`

Must enforce

- proof types are registered
- proof pointers resolve
- redaction rules apply

3.7 Kit Packager

Function: package the kit deterministically for downstream execution.

Includes

- canonical spec package
- resolved standards snapshot
- rendered templates + envelopes
- plan/work units (if included by profile)
- verification commands policy
- kit control plane (if shipped)
- manifest + entrypoint
- bundle export (zip/tar)

Produces

- `kit_manifest`
- `kit_entrypoint`
- `bundle_metadata`
- `bundle_export`

Must enforce

- kit completeness requirements (KIT)
- pinset embedded
- reproducibility metadata present

3.8 Operator Actions + Audit/Trace

Function: record operator actions, overrides, approvals.

Actions

- start/stop/pause/resume run
- rerun stage
- approve override
- change pins (if permitted)
- release bundle

Produces

- `audit_log_entries[]`
- `override_records[]` (when used)

Must enforce

- never-overridable gate list
- override expiry + scope requirements
- all actions link to `run_id`

ICP Output Artifacts

ICP Output Artifacts

O-00 Purpose

ICP output artifacts are the **auditable spine** of a run. They ensure:

- every decision is reproducible (pinset + selection outputs)
- every stage is traceable (stage reports + run log)
- every enforcement decision is explainable (gate report + evidence pointers)
- downstream execution is possible (kit outputs when applicable)
- failure states still leave usable diagnostics (reports still written on FAIL)

Always produced

O-01 `run_manifest`

Role: the canonical “run header” and index of what happened.

Must contain

- `run_id`
- normalized `run_context` (mode/profile/risk/executor/targets)
- resolved `pinset_ref` (or embedded pinset hash)
- stage list with status summary (PASS/FAIL/SKIP)
- artifact inventory (what artifacts exist and where)
- start/end timestamps (allowed noise, but required)
- links to: `run_log`, `stage_reports`, `gate_report`

Must be written

- at run start (initial manifest)
- updated after each stage transition (revisioned or updated with revision record)
- written even if run fails

O-02 `run_log`

Role: chronological trace of actions and decisions.

Must capture

- stage start/end events
- gate evaluation events (gate id + outcome)
- resolver decisions (standards/template selection pointers)
- verification command invocations and exit codes
- operator actions (pause/resume/rerun/override)

Format requirements

- append-only preferred
- each entry includes `timestamp`, `event_type`, `run_id`, and pointer refs to artifacts/logs

---

O-03 `stage_reports[]`

Role: per-stage structured "what was done + what was produced + what failed."

Each stage report must contain

- `run_id`, `stage_id`, `attempt_id` (if reruns exist)
- inputs consumed (artifact pointers)
- outputs produced (artifact pointers)
- validation results (schema checks, completeness checks)
- failures (if any): reason codes + evidence pointers + remediation list
- stage duration + status (PASS/FAIL/SKIP)

Rerun rule

- reruns produce a new stage report with a link to prior attempt(s)

O-04 `gate_report` (even if failing)

Role: authoritative pass/fail ledger for gate enforcement.

Must contain

- overall status (PASS/FAIL)
- per-gate results:
  - `gate_id`
  - predicate results (PASS/FAIL per predicate)

    - required evidence types satisfied or missing
    - evidence pointers
    - remediation list
- active gate set profile ID (what gate profile was used)
- run context summary (so report is self-contained)

Failure requirement

- on FAIL, report must still be complete and point to evidence/logs needed to debug

---

O-05 `pinset/lockfile`

Role: makes the run reproducible by freezing versions.

Must contain

- resolved versions for:
    - standards snapshot
    - templates used
    - gate set profile
    - schema registry versions (or schema ids/versions used)
    - toolchain versions (where applicable)
- hashes or version identifiers for each registry used
- resolution policy reference (if pins were resolved dynamically)

Requirement

- must exist before any stage that depends on versioned artifacts proceeds

---

Produced when relevant

O-06 `standards_snapshot` (+ applicability output)

Produced when

- any stage requires standards enforcement (normal builds; many modes)

`standards_applicability_output` must contain

- inputs used for applicability decision (run context fields)
- selected standards/packs + rationale tokens

- excluded standards/packs (optional, but useful)

`resolved_standards_snapshot` must contain

- resolved standard IDs + versions
- precedence/conflict decisions (what won, why)
- resolver order trace (or reference to resolver rules)
- snapshot hash for integrity and pinning

O-07 `template_selection_result` (+ `render_envelopes[]`)

Produced when

- templates are selected and rendered (TMP active)

`template_selection_result` must contain

- chosen template IDs + versions
- dependencies included (if any)
- selection reasoning fields tied to run context + standards snapshot
- selection hash for pinning

`render_envelopes[]` must contain

- template_id + version
- rendered output payload
- completeness status + missing fields list (if any)
- knowledge citations (if applicable)
- links to inputs used (canonical spec, standards snapshot)

O-08 `proof_ledger` (+ proof objects + verification results)

Produced when

- verification and proof capture are active (VER active)

`verification_run_result` must contain

- command list executed + versions
- exit codes
- log pointers

- environment fingerprint (allowed noise but recorded)

`proof_objects[]` must contain

- proof_id + proof_type
- source pointer (log/file/commit)
- applies_to (spec items / units / gates / run)
- hashes and redaction flags

`proof_ledger` must

- link proof_ids to run_id + gate_ids + affected targets
- be append-only or revisioned with explicit history

O-09 `kit_manifest` (+ entrypoint + bundle export)

Produced when

- KIT stage is active (kit packaging required by profile/mode)

`kit_manifest` must contain

- kit_id + run_id + pinset reference
- inventory of included artifacts with pointers + hashes
- required artifacts checklist (pass/fail)
- kit export metadata (bundle id, hash)

`entrypoint` must contain

- reading order (what build agent reads first)
- execution instructions (what to run, in what order)
- references to plan units and verification policy (if included)

`bundle_export` must

- be reproducible from the same pinset and inputs
- have a bundle hash recorded in metadata

O-10 `state_snapshot` (and/or rollback pointer)

Produced when

- STATE stage is active, or mode requires continuity/rollback readiness

`state_snapshot` must contain

- run_id + current stage statuses
- artifact inventory pointers
- last-known-good pointers (if available)
- rollback pointer (if rollback is supported/enabled)

Rollback pointer must

- reference a specific prior snapshot/bundle
- be resolvable to real artifacts (no dangling refs)

Determinism guarantees

Determinism guarantees

D-01 Deterministic input model

ICP must treat “same inputs” as a **canonical input set**. Determinism is evaluated against:

- `normalized_input` (not raw submission)
- the resolved `pinset/lockfile`
- the resolved `run_context` after normalization (mode/profile/risk/executor/targets)

Required normalizations

- stable ordering for arrays/sets (targets, domains, tags)
- stable casing/formatting for enums/IDs
- removal or isolation of any environment-dependent fields (machine paths, usernames, hostnames)

D-02 Deterministic selection outputs

Given identical canonical inputs and pinset, ICP must produce identical:

Standards selection and snapshot

- identical **applicability result**:
  - same selected standard IDs
  - same selected pack IDs (if applicable)
  - same rationale keys (based on deterministic criteria)
- identical **resolved standards snapshot**:
  - same standard versions
  - same precedence/conflict outcomes
  - same final resolved constraints

Template selection

- identical **template selection result**:
  - same template IDs + versions
  - same dependencies included
  - same selection reasoning fields (based on deterministic criteria)

D-03 Deterministic rendering outputs

Given identical canonical spec + standards snapshot + template selection:

- rendered envelopes must match in **all semantic fields**
- placeholder expansion must be deterministic:
    - derives must use deterministic sources only (canonical + snapshot + fixed registries)
    - tie-breaking rules must be defined for any “choose one” behavior

Render determinism constraints

- stable object key ordering in serialized output (or canonical JSON writer)
- stable list ordering for generated lists (fields, endpoints, screens, etc.) using declared sort keys
- deterministic formatting rules (whitespace, casing, numbering)

D-04 Allowed nondeterminism isolation

ICP may produce nondeterministic values only inside an **explicitly allowed noise surface**.

Allowed noise fields (examples of what must be isolated)

- `run_id`
- timestamps (`created_at`, `generated_at`, `updated_at`)
- host/environment fingerprints (`hostname`, `os_version`, `ci_run_id`)
- transient paths (`/tmp/...`)
- random seeds (must be fixed or recorded)

Noise handling rules

- noise fields must be:
    - confined to declared fields
    - never used in selection, ordering, or derivation
    - excluded from golden comparisons via normalization

D-05 Stage order invariance

Stage order must be fixed by:

- the pipeline definition (ORD/SYS stage order contract)

- activation rules derived deterministically from mode/profile

Requirements

- stage activation is a pure function of run context + registries
- if a stage is skipped, skip decision must be deterministic and recorded with:
    - `stage_id`
    - `skip_reason_code`
    - `mode/profile rule reference`

D-06 Resolver order invariance

Any resolver that merges, selects, or prioritizes must have a deterministic order:

Standards resolution order

- pack precedence order is explicit
- within a pack, standard evaluation order is explicit
- tie-breakers are explicit (e.g., severity > effective_date > id)

Template selection order

- deterministic template ranking rules
- deterministic dependency resolution order
- tie-breakers explicit (e.g., specificity > priority > id)

D-07 Deterministic pathing contract

ICP must write artifacts to deterministic paths derived from:

- `run_id` (for run-scoped storage)
- stable artifact IDs (for artifact-scoped storage)

Requirements

- path construction rules are centralized (no ad-hoc path building)
- the same artifact type always maps to the same relative path pattern
- overwrites are forbidden unless:
    - versioned by attempt id, or
    - explicitly revisioned with prior link

D-08 Deterministic comparisons (golden normalization rules)

When comparing outputs to goldens (or comparing two runs):

- comparisons must be performed on **normalized artifacts**
- normalization must:
    - remove/ignore allowed noise fields
    - canonicalize ordering (arrays, object keys)
    - canonicalize formatting (whitespace normalization where applicable)

Comparison outputs

- a diff report must identify:
    - which artifact differs
    - which fields differ
    - whether differences are within the allowed noise surface (must be classified)

D-09 Determinism evidence

Each run must emit evidence that determinism can be checked:

- pinset present and complete
- selection outputs recorded (standards + templates)
- artifact hashes recorded for key outputs (snapshot, selection result, render envelopes)

If any of these are missing, determinism is not provable for that run.

.

Failure semantics

Failure semantics (what “fail” means)

F-00 Purpose

A failure in ICP is not just “something went wrong.” It is a **contracted stop condition** that must still produce enough structured output to:

- identify the failing stage and rule
- explain *why* it failed
- point to evidence needed to debug
- preserve reproducibility context (pins + inputs)

F-01 Required artifacts on failure (always)

If any stage fails or any required gate fails, ICP must still write:

F-01.1 `gate_report` with remediation

- overall status = FAIL
- per-gate predicate results (PASS/FAIL per predicate)
- missing evidence types (if relevant)
- **remediation list** that is actionable (specific missing artifact, failing command, broken pointer)

F-01.2 Logs + evidence pointers

- command logs for any executed verification commands
- stage logs for the failing stage
- evidence pointers that resolve to:

    - file paths
    - log files
    - commit refs (if applicable)
    - artifact hashes (if applicable)

F-01.3 `stage_report` for the failing stage

Must include:

- `stage_id`
- `status = FAIL`
- failure reason code(s)
- inputs consumed (pointers)
- outputs produced (pointers; may be partial)
- validation results (what check failed)
- remediation list
- link to the `gate_report` section relevant to the failure (if gate-triggered)

F-01.4 `run_manifest` update

- run status must reflect failure
- manifest must link to the failing stage report and gate report
- pinset reference must remain intact

F-02 Failure classification (deterministic)

ICP must classify failures into one of these categories (minimum):

F-02.1 Contract failures (hard stop)

Failures that invalidate downstream correctness:

- schema invalid for required artifact
- missing required artifact
- pinset missing/invalid
- registry integrity failure (IDs missing, invalid versions)
- evidence pointer resolution failure for required pointers (when enforced)
- non-overridable gate failure

Action: fail-fast and stop progression immediately.

F-02.2 Verification failures (hard stop unless profile says otherwise)

- required command fails (test/lint/build/typecheck)
- proof requirements unmet for required gates

Action: stop at the verification/gate boundary, output proof/log pointers and remediation.

F-02.3 Recoverable execution failures (may allow rerun)

- transient tool errors (network fetch if allowed)
- flaky tests (if profile defines a rerun policy)
- temporary resource exhaustion

Action: may transition to rerun policy, but must still write an attempt report and preserve prior attempt outputs.

F-03 Fail-fast rules (when to stop immediately)

ICP must stop immediately when any of these occur:

- a required schema validation fails for a stage output
- a required input artifact is missing for the next stage
- pinset cannot be resolved or validated
- registry load/validation fails
- a required gate fails with no override path
- artifact paths/pointers required for gating do not resolve

F-04 Rerun behavior on failure (attempt-safe)

If rerun is allowed by mode/profile:

- failure must still be recorded as an attempt with its own stage report
- rerun creates a new `attempt_id`
- outputs from prior attempt must not be overwritten
- rerun must link to prior attempt(s) (attempt chain)

F-05 Override interaction (if enabled)

If an override is permitted:

- the failing gate must be overridable (not in never-override list)
- override must be recorded with:

    - scope, expiry, approver, reason
- after override, ICP may proceed, but must:
    - keep the original gate failure recorded
    - mark run as “proceeded with override” in manifest/state

---

F-06 Minimal failure deliverable

A run is considered “failed correctly” only if:

- the failing stage is identified
- the failing rule/gate is identified
- evidence pointers resolve
- remediation steps are included
- pinset and run context are preserved for repro/debug

Hard boundaries

Hard boundaries (locked)

B-00 Purpose

Hard boundaries prevent role confusion across the three control planes and keep ICP deterministic, auditable, and safe. These are **non-negotiable constraints**.

B-01 ICP does not modify target app repos directly

Meaning

- ICP never writes to or edits the target application repository as a “build action.”

Allowed

- generating an agent kit bundle that *will later* be used to modify a target repo
- writing ICP artifacts to AXION-controlled storage paths (runs, proofs, bundles)

Not allowed

- committing code changes to the target repo
- running refactors/upgrades in the target repo as part of ICP execution

Enforcement expectation

- path allowlist: ICP can write only under AXION-controlled directories
- external side effects are blocked unless explicitly in the AXION runtime boundary

---

B-02 ICP does not invent standards/templates/gates

Meaning

- ICP can only select from registered items with valid IDs and versions.

Allowed

- selecting standards/templates/gates based on applicability/profile rules
- emitting selection outputs that record which registered items were used

Not allowed

- creating ad-hoc standards or gate predicates on the fly
- generating new template definitions and treating them as “official” without registry entry

Enforcement expectation

- selection must fail if an ID is referenced but not found
- predicate evaluation must fail if an operator is not in the operator registry

---

B-03 ICP does not accept unverifiable success

Meaning

- “PASS” is only valid when required evidence exists and is linked.

Allowed

- using proof types defined in the proof registry
- using evidence sources allowed by evidence source registry
- allowing optional proofs only if policy explicitly marks them optional

Not allowed

- passing a gate without required proof artifacts

- marking a stage complete without its required outputs and validations

Enforcement expectation

- evidence policy mapping (gate → required proof types) is mandatory for required gates
- missing proof is a hard stop for PASS in required profiles

B-04 ICP must write artifacts that downstream systems can validate without context

Meaning

- Kits and reports must be self-describing: downstream execution should not need “tribal knowledge” or internal operator memory.

Required properties

- every artifact references:
    - schema version
    - IDs/versions of standards/templates/gates used (directly or via pinset)
    - pointers to inputs and evidence
- entrypoint and manifest define reading order and required contents

Enforcement expectation

- kit validation must be possible using only:
    - the kit itself
    - the included pinset/manifest
    - included gate reports/proofs (as applicable)

- missing context must be treated as failure, not guessed around

Kit Control Plane

External Control Plane / Kit Control Plane (KCP) — Detailed Spec

Role

Lives: inside the Agent Kit (ships with the kit)
Primary user: build agent executing the kit
Purpose: enforce kit-local rules during the build and ensure work + proof stay aligned to AXION contracts.

---

1) KCP Inputs (what it consumes)

1.1 Kit artifacts (required)

- `kit_manifest`
- `kit_entrypoint`
- `pinset/lockfile` (embedded or referenced)
- `standards_snapshot` (if included by profile)
- `render_envelopes[]` (template outputs)
- `verification_policy` (required commands + settings)
- `plan_units[]` (PLAN_UNIT files) when planning is included

1.2 Execution context (required)

- executor identity (internal/external)
- environment descriptor (ci/local; OS; tool versions)
- repo target path (where the build will occur)

1.3 Optional inputs

- prior kit-local run reports (for resume)
- prior RESULT/proof entries (for incremental work)

---

2) KCP State Model (kit-local lifecycle)

2.1 Kit-run states (minimum)

- `READY` → `EXECUTING` → `VERIFYING` → (`BLOCKED` | `FAILED` | `COMPLETE`)

Optional

- `PAUSED`, `CANCELLED`, `RESUMING`

2.2 Work-unit states (minimum)

- `NOT_STARTED`, `IN_PROGRESS`, `DONE`, `FAILED`, `SKIPPED`

2.3 Transition rules (deterministic)

- cannot start execution unless kit validation passes
- cannot mark unit DONE unless required acceptance outputs exist:
  - implementation refs present
  - proof refs present
  - (if required) local verify results captured
- unit ordering must follow the kit's plan sequencing rules (if provided)
- resume must preserve prior unit attempts and produce new attempt records

---

3) KCP Runtime Modules (what it actually does)

3.1 Kit Validator

Function: validate kit integrity before any build work begins.

Validates

- required artifacts present
- schemas valid for: manifest/entrypoint/envelopes/snapshot (as applicable)
- entrypoint compliance (reading order + required instructions exist)
- pinset present and parseable

Produces

- `kit_validation_report`

Must enforce

- no missing required kit artifacts
- schema versions match pinset expectations

3.2 Work Unit Manager (1-target enforcement)

Function: read plan units, enforce “1 target per unit,” and guide next work.

Responsibilities

- load `PLAN_UNIT_*`
- validate unit schema + enforce `targets.length == 1`
- recommend next unit (deterministic ordering rules)
- track unit status + attempts

Produces

- `unit_status_index`
- optional `next_unit_recommendation`

Must enforce

- reject malformed or multi-target units (hard stop or quarantine)
- preserve attempt history on reruns

3.3 Local Verification Runner

Function: run required verification commands locally (lint/test/build/typecheck) and capture logs.

Responsibilities

- execute commands from `verification_policy`
- capture stdout/stderr and exit codes
- write deterministic log paths
- normalize results into structured command results

Produces

- `verification_run_result`
- `logs/<command_id>.log`

Must enforce

- required commands must run (unless policy marks optional)
- command results must be recorded even on failure

3.4 Result Writer / Ingestion Helper

Function: produce RESULT artifacts that AXION can ingest without guesswork.

Responsibilities

- create `RESULT_<UNIT_ID>.json` templates
- enforce required fields:
  - `implementation_refs[]`
  - `proof_refs[]`
- attach pointers to logs and artifacts as proof refs
- archive or mark completed results deterministically

Produces

- `RESULT_<UNIT_ID>.json`
- optional `result_index`

Must enforce

- a unit cannot be marked DONE without non-empty `implementation_refs` and `proof_refs`

3.5 Proof Capture + Normalization

Function: generate proof objects that conform to AXION proof type registry.

Responsibilities

- map verification outputs to proof objects
- create proof entries that link:
  - unit_id

    - target (spec item)
    - gate ids (if known)
    - run context (kit-local run id)

Produces

- `proof_objects[]`
- optional `proof_ledger` (kit-local)

Must enforce

- proof types used must be allowed by the kit's proof registry snapshot
- pointers must resolve (logs/files exist)

3.6 Execution Guardrails (policy enforcement)

Function: enforce safety boundaries during build execution.

Enforces

- forbidden path access rules (no touching restricted areas)
- external agent restrictions (if executor is external)
- plagiarism/reuse policy constraints (no code-donor behavior without allowlist)
- secrets/PII redaction rules for logs and outputs

Produces

- `guardrail_report` (violations + pointers)

Must enforce

- violations produce FAIL/BLOCKED with remediation and evidence pointers

---

3.7 Kit-local Run Report

Function: produce a mergeable report that ICP/MCP can ingest.

Includes

- kit validation result
- executed units + statuses + attempts

- RESULT files produced
- verification command results + log pointers
- proofs produced + pointers
- guardrail violations (if any)

Produces

- `kit_run_report`

Must enforce

- report is self-contained and references all required artifacts via pointers

---

4) KCP Output Artifacts (the contract it guarantees)

Always produced

- `kit_validation_report`
- `kit_run_report` (even if failing, partial but structured)
- logs for any executed verification commands
- unit status tracking artifact (`unit_status_index` or equivalent)

Produced when relevant

- `RESULT_<UNIT_ID>.json` files
- proof objects / kit-local proof ledger (if enabled)
- guardrail report (if any checks run)

---

5) Determinism guarantees (kit-local)

KCP must guarantee:

- same kit + same plan units + same policy → same unit ordering decisions
- log and report paths follow deterministic patterns

- allowed nondeterminism (timestamps, local run id) is isolated and excluded from comparisons

---

6) Failure semantics

When KCP fails:

- it must still emit:
  - `kit_run_report` with failing unit/stage and remediation
  - log pointers to failing command outputs
  - any partial RESULT/proof artifacts produced so far
- it must fail-fast on:
  - missing kit artifacts
  - schema invalid artifacts
  - multi-target units (violates 1-target rule)
  - guardrail violations that are non-overridable for the kit

---

7) Hard boundaries (locked)

- KCP does not change AXION registries or standards; it executes only within kit-defined artifacts and policies.
- KCP optimizes build correctness + evidence, not system design or policy authoring.
- KCP must not expand scope beyond the kit's declared targets and allowed actions.

KCP Inputs Contracts

KCP Inputs (what it consumes)

1.1 Kit artifacts (required)

Purpose: everything KCP needs must be present *inside the kit* (or referenced by deterministic pointers) so execution does not depend on external context.

- `kit_manifest`
  - authoritative inventory of what's in the kit
  - includes hashes/pointers for key artifacts
- `kit_entrypoint`
  - ordered instructions for what to read/execute first
  - references required artifacts and run expectations
- `pinset/lockfile` *(embedded or referenced)*
  - freezes versions of: templates, standards snapshot, gates (if shipped), schemas, tooling expectations
  - required for reproducible behavior
- `standards_snapshot` *(if included by profile)*
  - the resolved ruleset the build must obey
  - used for guardrails and validation
- `render_envelopes[]` *(template outputs)*
  - the filled template artifacts KCP will rely on during build
  - includes completeness status and citations (if required)
- `verification_policy`
  - required commands + settings (lint/test/build/typecheck)
  - evidence requirements for proofs (what outputs must be captured)
- `plan_units[]` *(PLAN_UNIT files, when planning is included)*
  - atomic work instructions, enforced as 1-target per unit
  - include acceptance criteria and expected outputs

Determinism requirements

- all required kit artifacts must be discoverable via `kit_manifest` inventory
- file naming/pointer rules must allow deterministic lookup (no "guess the filename")
- schema versions referenced by artifacts must be resolvable from the pinset

---

1.2 Execution context (required)

Purpose: KCP must bind its work to a concrete execution environment and target repo location.

- **executor identity** *(internal | external)*
    - selects access constraints and guardrail strictness
- **environment descriptor**
    - `execution_mode` *(ci | local)*
    - OS/platform
    - tool versions (node, package manager, compiler/test runner versions if relevant)
    - optional environment fingerprint (container hash, runner id)
- **repo target path**
    - the filesystem root where the build agent will apply changes
    - must be validated against allowed path rules

Determinism requirements

- environment descriptor is recorded, but must not influence selection/render decisions (only enforcement)
- repo path must be normalized and restricted to allowed roots

---

1.3 Optional inputs

Purpose: allow resume/incremental runs without losing auditability.

- **prior kit-local run reports** *(for resume)*
    - used to detect completed units and preserve attempt history
- **prior RESULT/proof entries** *(for incremental work)*
    - used to avoid duplicate work and to maintain continuity of evidence

Determinism requirements

- resume must not overwrite prior artifacts; it must create new attempts or revision records
- incremental inputs must be validated (schema + pointer resolution) before use

KCP State Model

KCP State Model (kit-local lifecycle)

2.1 Kit-run states (minimum)

State sequence

- **READY** → **EXECUTING** → **VERIFYING** → (**BLOCKED** | **FAILED** | **COMPLETE**)

Optional states

- **PAUSED**
- **CANCELLED**
- **RESUMING**

State meaning (contract)

- **READY:** kit is loaded; preflight checks not yet completed or just completed
- **EXECUTING:** work units are being performed (code changes, file edits, implementation work)
- **VERIFYING:** verification_policy commands are executing and proof is being captured
- **BLOCKED:** execution cannot proceed due to a fixable constraint (missing artifact, failing gate that can be remedied, missing proof refs)
- **FAILED:** execution cannot proceed due to a non-recoverable failure (non-overridable violation, invalid kit, repeated verify failures per policy)
- **COMPLETE:** all required work units complete and required verification/proofs captured
- **PAUSED:** operator/agent intentionally paused; state must be resumable
- **CANCELLED:** explicitly terminated; artifacts remain for audit
- **RESUMING:** transitional state used to load prior attempt data and re-enter EXECUTING or VERIFYING deterministically

---

2.2 Work-unit states (minimum)

- **NOT_STARTED**
- **IN_PROGRESS**
- **DONE**
- **FAILED**
- **SKIPPED**

Work-unit meaning (contract)

- **NOT_STARTED:** unit exists and is eligible but not begun
- **IN_PROGRESS:** unit is actively being worked; partial outputs may exist
- **DONE:** unit meets acceptance requirements and has recorded implementation + proof refs
- **FAILED:** unit failed and cannot proceed without rerun/new attempt
- **SKIPPED:** unit not executed because plan sequencing/mode explicitly allows skip (must record reason)

2.3 Transition rules (deterministic)

Kit-run transition rules

- **READY → EXECUTING**
    - allowed only if **kit validation passes**
    - kit validation output must be recorded and referenced
- **EXECUTING → VERIFYING**

    - allowed only when:
        - current unit attempt produces required code/output artifacts, and
        - verification_policy indicates verification is required at this point (per unit or per batch)
- **VERIFYING → COMPLETE**
    - allowed only when:
        - required verification commands have PASS outcomes (as required by policy), and
        - required proof refs are written and resolvable
- **VERIFYING → BLOCKED**
    - allowed when:
        - verification fails in a recoverable way (policy allows retry), or
        - required proof refs are missing but can be added
- **EXECUTING/VERIFYING → FAILED**
    - allowed when:
        - kit artifact is invalid (schema fail, missing required file)
        - non-overridable guardrail violation occurs
        - verification fails and policy does not allow retry/override
        - pointer resolution fails for required evidence
- **EXECUTING/VERIFYING → PAUSED**
    - operator/agent action; must record who/why/when and preserve state
- **PAUSED → RESUMING → EXECUTING/VERIFYING**
    - resume must load last known unit attempt deterministically

- **EXECUTING/VERIFYING → CANCELLED**
    - operator/agent action; must preserve artifacts and record cancellation reason

---

Work-unit transition rules

- **NOT_STARTED → IN_PROGRESS**
    - allowed only if:
        - unit schema is valid
        - `targets.length == 1`
        - unit is eligible under sequencing rules (if sequencing provided)
- **IN_PROGRESS → DONE**
    - allowed only if **all** required acceptance outputs exist and validate:
        - `implementation_refs[]` present and non-empty
        - `proof_refs[]` present and non-empty
        - if verification is required by policy: verification results captured and referenced
    - DONE must also record evidence pointers that dereference successfully
- **IN_PROGRESS → FAILED**
    - allowed when:
        - required outputs cannot be produced due to hard failure
        - verification required and fails with no retry allowed
    - must record failure reason + pointers to logs/evidence
- **NOT_STARTED/IN_PROGRESS → SKIPPED**

    - allowed only if:
        - plan sequencing rules or mode/profile explicitly allow skip
    - must record skip reason code and authority (rule reference)

Ordering rules (plan sequencing)

- If the kit includes sequencing rules:
    - KCP must follow the deterministic order given
- If the kit does not include explicit sequencing:
    - KCP must use a deterministic fallback order (e.g., by unit_id sort)
- KCP must never reorder units “based on intuition”

Resume rules (attempt preservation)

- resume must not overwrite previous attempts
- each resume creates either:
    - a new attempt id for the current unit, or
    - an explicit continuation record linked to the prior attempt
- attempt chains must remain readable and auditable in the kit-local report

KCP Runtime Modules

KCP Runtime Modules (what it actually does)

Each responsibility maps to a module that produces artifacts.

3.1 Kit Validator

Function: validate kit integrity before any build work begins.

Validates

- required artifacts present
- schemas valid for: manifest / entrypoint / envelopes / snapshot (as applicable)
- entrypoint compliance (reading order + required instructions exist)
- pinset present and parseable

Produces

- `kit_validation_report`

Must enforce

- no missing required kit artifacts
- schema versions match pinset expectations

3.2 Work Unit Manager (1-target enforcement)

Function: read plan units, enforce “1 target per unit,” and guide next work.

Responsibilities

- load `PLAN_UNIT_*`

- validate unit schema + enforce `targets.length == 1`
- recommend next unit (deterministic ordering rules)
- track unit status + attempts

Produces

- `unit_status_index`
- optional `next_unit_recommendation`

Must enforce

- reject malformed or multi-target units (hard stop or quarantine)
- preserve attempt history on reruns

3.3 Local Verification Runner

Function: run required verification commands locally (lint/test/build/typecheck) and capture logs.

Responsibilities

- execute commands from `verification_policy`
- capture stdout/stderr and exit codes
- write deterministic log paths
- normalize results into structured command results

Produces

- `verification_run_result`

- `logs/<command_id>.log`

Must enforce

- required commands must run (unless policy marks optional)
- command results must be recorded even on failure

3.4 Result Writer / Ingestion Helper

Function: produce RESULT artifacts that AXION can ingest without guesswork.

Responsibilities

- create `RESULT_<UNIT_ID>.json` templates
- enforce required fields:
  - `implementation_refs[]`
  - `proof_refs[]`
- attach pointers to logs and artifacts as proof refs
- archive or mark completed results deterministically

Produces

- `RESULT_<UNIT_ID>.json`
- optional `result_index`

Must enforce

- a unit cannot be marked DONE without non-empty `implementation_refs` and `proof_refs`

3.5 Proof Capture + Normalization

Function: generate proof objects that conform to AXION proof type registry.

Responsibilities

- map verification outputs to proof objects
- create proof entries that link:
    - `unit_id`
    - target (spec item)
    - gate ids (if known)
    - run context (kit-local run id)

Produces

- `proof_objects[]`
- optional `proof_ledger` (kit-local)

Must enforce

- proof types used must be allowed by the kit's proof registry snapshot
- pointers must resolve (logs/files exist)

3.6 Execution Guardrails (policy enforcement)

Function: enforce safety boundaries during build execution.

Enforces

- forbidden path access rules (no touching restricted areas)

- external agent restrictions (if executor is external)
- plagiarism/reuse policy constraints (no code-donor behavior without allowlist)
- secrets/PII redaction rules for logs and outputs

Produces

- `guardrail_report` (violations + pointers)

Must enforce

- violations produce FAIL/BLOCKED with remediation and evidence pointers

---

3.7 Kit-local Run Report

Function: produce a mergeable report that ICP/MCP can ingest.

Includes

- kit validation result
- executed units + statuses + attempts
- RESULT files produced
- verification command results + log pointers
- proofs produced + pointers
- guardrail violations (if any)

Produces

- `kit_run_report`

Must enforce

- report is self-contained and references all required artifacts via pointers

KCP Output Artifacts

KCP Output Artifacts

O-00 Purpose

KCP outputs are the **kit-local evidence spine**. They make the build agent's work auditable and ingestible by ICP/MCP without additional context.

Always produced

O-01 `kit_validation_report`

Role: preflight proof that the kit was valid before execution.

Must include

- kit identity: `kit_id` (or bundle id), `run_id` (if provided), pinset reference/hash
- validation results by category:
  - required artifacts present
  - schema validation results (manifest/entrypoint/envelopes/snapshot)
  - entrypoint compliance checks
  - pinset parse/consistency checks
- PASS/FAIL summary
- evidence pointers to any failing artifacts/logs
- tool/environment descriptor used for validation

Written when

- before any unit execution begins
- written even if validation fails

O-02 `kit_run_report` (even if failing; partial but structured)

Role: the primary "handoff" artifact back to ICP/MCP.

Must include

- kit identity and execution context:
  - executor identity (internal/external)
  - environment descriptor (ci/local, OS, tool versions)

    - repo target path (normalized)
- overall kit-run status: READY/EXECUTING/VERIFYING/BLOCKED/FAILED/COMPLETE
- unit summary:
    - list of units discovered
    - per-unit status, attempt chain links
    - pointers to `RESULT_<UNIT_ID>.json` (if produced)
- verification summary:
    - command results + exit codes
    - log pointers
- proof summary:
    - proof object pointers (if produced)
- guardrail summary:
    - any violations + pointers
- remediation block on failure:
    - why it stopped
    - what to fix next

Written when

- updated at minimum on state transitions and at end
- must exist even on early failure (e.g., kit validation fail)

O-03 Logs for any executed verification commands

Role: raw evidence for verification outcomes.

Must include

- stdout/stderr capture (or pointer to it)
- exit code
- start/end timestamps
- command id that maps to verification_policy

Pathing

- deterministic path scheme (e.g., `logs/<command_id>.log`) plus attempt id if rerun

---

O-04 Unit status tracking artifact (`unit_status_index` or equivalent)

Role: machine-readable unit progress tracker used by:

- KCP resume logic
- ICP/MCP ingestion
- “next unit” recommendation

Must include

- unit_id list
- for each unit:
  - target (category:id) (1-target rule)
  - state (NOT_STARTED/IN_PROGRESS/DONE/FAILED/SKIPPED)
  - current attempt id + prior attempt refs
  - pointers to RESULT/proof/log artifacts
- ordering/selection metadata:
  - ordering rule used
  - next recommended unit (optional)

Produced when relevant

O-05 `RESULT_<UNIT_ID>.json` files

Role: standardized handoff of completion evidence per work unit.

Must include

- `unit_id`
- `submitted_at`
- `implementation_refs[]` (non-empty for DONE)
- `proof_refs[]` (non-empty for DONE)
- optional summary/notes
- pointers that resolve to real logs/files/commits

Requirement

- cannot be considered DONE without valid RESULT content

O-06 Proof objects / kit-local proof ledger (if enabled)

Role: structured proof aligned to AXION proof types.

Proof objects must include

- `proof_id`, `proof_type`
- applies_to: unit_id + target and/or gate id
- source pointers (logs/files/hashes)
- environment/tool metadata + redaction flags

Kit-local proof ledger

- append-only or revisioned
- indexes proof objects for ingestion

O-07 `guardrail_report` (if guardrails executed)

Role: records any policy violations and blocks/limits actions.

Must include

- violation list with:
  - rule_id
  - severity
  - affected paths/artifacts
  - evidence pointers
  - remediation steps
- whether violation is overridable (if kit policy allows)
- final guardrail status (PASS/BLOCKED/FAIL)

Failure interaction

- non-overridable violations must force kit-run state to FAILED
- overridable violations must force kit-run state to BLOCKED until resolved/recorded

KCP Determinism guarantees

Determinism guarantees (kit-local)

D-00 Purpose

KCP determinism ensures that kit execution is **repeatable and audit-safe**: the same kit and inputs produce the same operational decisions and comparable artifacts.

D-01 Deterministic “input set” definition

KCP evaluates determinism against this canonical set:

- `kit_manifest` (as the authoritative inventory)
- `kit_entrypoint`
- `pinset/lockfile`
- `verification_policy`
- `plan_units[]` (when present)
- normalized execution context fields that are allowed to influence behavior:
  - `executor_identity` (internal/external)
  - `execution_mode` (ci/local)

All other environment specifics are recorded but must not alter selection/ordering logic.

D-02 Deterministic unit ordering

Given the same:

- kit
- plan units
- ordering rules (from entrypoint/plan policy)
- executor identity constraints

KCP must produce the same:

- unit eligibility set
- unit order
- “next unit” recommendation

Ordering rules

- If an explicit plan sequencing rule is provided in the kit, it is authoritative.
- If not provided, KCP uses a deterministic fallback rule:
    - stable sort key (e.g., `unit_id` lexical sort)
    - tie-breakers explicitly defined (never “pick one randomly”)

D-03 Deterministic enforcement decisions

For the same kit and the same unit state:

- `targets.length == 1` enforcement must always yield the same decision (PASS/FAIL)
- acceptance requirements (implementation_refs/proof_refs/verify outputs) must always evaluate the same way
- guardrail rules must produce the same violations for the same actions

---

D-04 Deterministic artifact pathing

Logs and reports must follow deterministic patterns derived from stable IDs:

Path inputs

- `unit_id`
- `attempt_id`
- `command_id`
- kit-local run id (allowed noise; used only to scope paths if required)

Requirements

- no ad-hoc filenames
- reruns must not overwrite prior logs; they must write under a new attempt scope
- reports must always be written to the same relative locations for the same artifact types

---

D-05 Allowed nondeterminism isolation

Allowed nondeterminism must be confined to explicitly designated fields:

- timestamps (`created_at`, `submitted_at`, start/end times)
- local `kit_run_id` (if generated)
- environment fingerprints (runner id, hostname)

Rules

- nondeterministic fields must not affect ordering or validation outcomes
- golden comparisons must normalize out these fields before comparing

---

D-06 Determinism evidence

KCP must emit enough evidence to verify determinism:

- the ordering rule used (explicit sequencing vs fallback rule)
- per-unit attempt chain (so rerun behavior is auditable)
- command results + log pointers with deterministic naming
- kit_run_report containing normalized summary of decisions

KCP Failure semantics

Failure semantics (KCP)

F-00 Purpose

A KCP failure must be **auditable and recoverable**: it stops unsafe progress but still leaves a complete trail of what happened, why it stopped, and what evidence exists.

F-01 Required outputs on failure (always)

When KCP enters `FAILED` or `BLOCKED`, it must still emit:

F-01.1 `kit_run_report` (failure-complete)

Must include:

- final kit-run state (`BLOCKED` or `FAILED`)
- failing location:
  - `unit_id` (if unit-scoped)
  - `phase` (`KIT_VALIDATION` | `EXECUTING` | `VERIFYING` | `GUARDRAILS`)
- failure reason code(s)
- remediation steps (actionable)
- evidence pointers to:
  - failing artifacts
  - failing logs
  - any guardrail report section (if relevant)

F-01.2 Log pointers to failing command outputs

If any verification command was executed:

- command id
- exit code
- pointer to stdout/stderr log artifact
- attempt id (if rerun)

F-01.3 Partial artifacts preserved

KCP must preserve and reference any partial outputs that exist:

- partially filled `RESULT_<UNIT_ID>.json` (if created)

- proof objects (if created)
- intermediate validation reports (kit_validation_report, guardrail_report)

No partial artifact may be overwritten during failure handling.

F-02 Fail-fast conditions (hard stops)

KCP must fail-fast immediately (no further unit execution) when any of the following occur:

F-02.1 Missing kit artifacts

- required kit artifacts absent from manifest inventory
- required files referenced by entrypoint do not exist
- pinset missing when required by policy

F-02.2 Schema invalid artifacts

- `kit_manifest` schema invalid
- `kit_entrypoint` schema invalid
- required render envelope schema invalid
- standards snapshot schema invalid (when included)
- verification policy schema invalid

F-02.3 Multi-target plan units (1-target rule violation)

- any unit has `targets.length != 1`
- any unit target missing required fields (`category`, `id`)

Disposition rule

- either hard stop, or quarantine the unit, depending on kit policy
- if quarantined, quarantine must be recorded in the run report with reason

F-02.4 Non-overridable guardrail violations

Examples (kit-defined):

- forbidden path access
- secrets/PII detected in logs or outputs
- plagiarism/reuse rule violation without allowlist
- external agent attempts restricted access

These must force `FAILED` (not `BLOCKED`) unless policy explicitly allows an override workflow.

F-03 BLOCKED vs FAILED (deterministic classification)

KCP must classify failures as:

BLOCKED (recoverable)

- missing proof refs that can be added
- verification failure where retry is allowed by policy
- unit failed due to incomplete implementation but no policy violation

FAILED (non-recoverable)

- invalid kit artifacts or schemas
- non-overridable guardrail violation
- repeated verification failure beyond allowed retries
- pointer resolution failure for required evidence (when enforced)

Classification must be recorded in the kit_run_report as a reason code, not just a free-text message.

F-04 Attempt handling on failure

If a unit fails:

- unit state becomes `FAILED`
- the attempt is closed and recorded
- rerun (if permitted) creates a new attempt id and must link to the failed attempt
- prior attempt logs/results remain immutable and referenced

F-05 Minimal failure correctness criteria

A KCP failure is considered “well-formed” only if:

- failing phase and (if applicable) unit_id are identified
- remediation steps are present

- evidence pointers resolve
- prior outputs and attempts are preserved and referenced

KCP Hard boundaries

Hard boundaries (locked) — KCP

B-00 Purpose

KCP boundaries keep the kit execution plane **safe, minimal, and non-authoritative**. KCP is an executor-aligned enforcer, not a system author.

B-01 KCP does not change AXION registries or standards

Meaning

- KCP treats standards, templates, gates, proof types, and policies shipped in the kit as **read-only inputs**.

Allowed

- reading and validating registry snapshots included in the kit (if present)
- writing kit-local artifacts (RESULT files, logs, kit_run_report, proof objects) as outputs

Not allowed

- editing or regenerating standards definitions, template definitions, gate definitions
- selecting “new standards” outside the kit snapshot
- modifying resolver rules or applicability behavior

Boundary enforcement expectation

- write-path allowlist: KCP writes only in kit-approved output directories
- any attempt to modify protected kit areas is a guardrail violation

---

B-02 KCP optimizes build correctness + evidence, not system design

Meaning

- KCP’s job is to **execute** the kit and prove outcomes, not to decide what the kit should have contained.

Allowed

- applying plan units to implement targets
- running required verification commands and capturing logs

- producing RESULT/proof artifacts that demonstrate compliance

Not allowed

- creating new templates or redefining template completeness rules
- inventing new gates or relaxing gate requirements
- rewriting the verification policy beyond what the kit specifies

Boundary enforcement expectation

- verification policy is treated as authoritative
- “missing rules” are reported as failures or blockers, not auto-patched by KCP

B-03 KCP must not expand scope beyond the kit’s declared targets and allowed actions

Meaning

- KCP is scoped to the explicit targets defined by the kit (platforms/domains/units).

Allowed

- file changes necessary to satisfy the active unit’s target
- running the commands listed in `verification_policy`
- writing outputs required by the unit acceptance criteria

Not allowed

- implementing extra features not represented by a plan unit or declared target
- touching unrelated modules “because it seems useful”
- running arbitrary commands outside policy
- accessing disallowed directories or external resources when restricted (especially for external agents)

Boundary enforcement expectation

- unit-to-changes trace: every meaningful change should map to an active unit_id
- command allowlist: only policy-declared commands may run
- external agent constraints must be strictly enforced (least privilege)

Maintenance Control Plane (MCP)

Maintenance Control Plane (MCP) — Detailed Spec

Role

Lives: inside the target repo, outside AXION (repo-resident assistant)
Primary user: operators/devs + maintenance agent(s)
Purpose: keep the repo healthy over time (upgrades, migrations, test hardening) while staying compatible with AXION contracts.

---

1) MCP Inputs (what it consumes)

1.1 Repo state (required)

- current repo source tree
- dependency manifests + lockfiles
- CI workflow definitions
- test suite configuration
- migration history (if present)
- versioning metadata (changelog, releases, tags)

1.2 AXION compatibility surfaces (required)

- AXION-required artifacts that this repo must keep compatible with (e.g., kit manifests, proof formats, gate reports) *as applicable*
- current MM-xx definitions or the maintenance-mode mapping used by this repo
- pin policies/toolchain constraints enforced by AXION for this repo

1.3 Maintenance request / intent (required)

- maintenance mode id (MM-xx) or maintenance intent type:
    - dependency upgrade, migration, refactor, test hardening, CI fix, etc.
- scope constraints (what areas are allowed to change)
- risk class (prototype/production/compliance) as it impacts strictness

1.4 Optional inputs

- prior maintenance reports and diffs
- prior failures (CI logs, gate reports)
- known issues/backlog references

---

2) MCP State Model (repo maintenance lifecycle)

2.1 Maintenance run states (minimum)

- **PLANNED** → **APPLYING** → **VERIFYING** → (**BLOCKED** | **FAILED** | **COMPLETE**)

Optional

- **PAUSED**, **CANCELLED**, **ROLLBACKING**

2.2 Work-unit states (minimum)

- **NOT_STARTED**, **IN_PROGRESS**, **DONE**, **FAILED**, **SKIPPED**

2.3 Transition rules (deterministic)

- cannot enter APPLYING unless:
  - scope constraints are validated
  - baseline health checks pass (or are explicitly waived by mode)
- cannot mark COMPLETE unless:
  - verification passes (tests/lint/build)
  - AXION compatibility checks pass (where applicable)
  - required maintenance report is produced
- rollback must preserve attempt history and produce a rollback record

---

3) MCP Runtime Modules (what it actually does)

3.1 Dependency Upgrade Manager

Function: bump deps safely and consistently.

Responsibilities

- apply version bumps according to policy (patch/minor/major rules)
- update lockfiles deterministically
- identify breaking changes (if detectable)
- record change notes

Produces

- `dependency_update_report`
- updated lockfiles
- changelog entry (if required)

Must enforce

- lockfile discipline (no drifting lock state)
- pinned toolchain requirements (if required)

3.2 Migration Manager

Function: manage schema/data evolution safely.

Responsibilities

- generate/apply migrations
- enforce migration ordering rules
- run backcompat checks (if required)
- create data repair scripts when needed

Produces

- `migration_plan`
- migration artifacts (files/scripts)
- `migration_verification_report`

Must enforce

- migration safety rules (no destructive changes without plan)
- rollback strategy recorded (if required by mode)

3.3 Test Suite Maintainer

Function: evolve tests to reduce drift and increase confidence.

Responsibilities

- add regression tests for known failures
- reduce flakiness (stabilize ordering/time assumptions)
- adjust coverage targets (if required by policy)

Produces

- `test_update_report`
- new/updated tests
- flake triage notes (if relevant)

Must enforce

- tests remain deterministic under pinned environment
- new tests map to failure causes (no “random” tests)

3.4 Refactor and Hygiene Manager

Function: restructure safely without breaking contracts.

Responsibilities

- apply controlled refactors (module boundaries, naming, folder moves)
- enforce lint/type fixes
- avoid breaking public interfaces unless change process followed

Produces

- `refactor_report`
- diff summary of moved/renamed components

Must enforce

- contract preservation (schemas, IDs, public interfaces)
- change-impact notes updated when required

3.5 CI Workflow Maintainer

Function: keep automation valid and aligned with system needs.

Responsibilities

- update workflows (steps, caching rules, tool versions)
- ensure doctor/verify remain enforced
- ensure artifacts are uploaded on failure

Produces

- `ci_update_report`
- updated CI files

Must enforce

- stable toolchain pins
- reproducibility checks preserved when enabled

3.6 AXION Integration Maintainer

Function: keep repo compatible with AXION contracts and ingestion.

Responsibilities

- validate that required AXION artifacts still match schemas
- validate that gate reports/proofs are still producible
- ensure kit outputs remain valid when this repo is a kit producer/consumer
- ensure compatibility when schemas/registries change (migration support)

Produces

- `axion_compatibility_report`

Must enforce

- schema compatibility rules
- version/pin compliance where required

3.7 Maintenance-mode Runner

Function: execute maintenance flows aligned to MM-xx modes.

Responsibilities

- interpret MM-xx for maintenance actions
- enforce mode-specific strictness and “never skip” rules
- enforce post-incident backfill (when mode requires)

Produces

- `maintenance_run_report`
- links to verification outputs and any override records

Must enforce

- mode rules are applied deterministically
- any override follows override policy and is recorded

---

4) MCP Output Artifacts (the contract it guarantees)

Always produced

- `maintenance_run_report`
- verification logs for executed commands
- status tracking for maintenance units (if unitized)

Produced when relevant

- `dependency_update_report`
- `migration_plan` + `migration_verification_report`
- `test_update_report`
- `refactor_report`
- `ci_update_report`
- `axion_compatibility_report`
- rollback record (if rollback executed)

---

5) Determinism guarantees (repo maintenance)

MCP must guarantee:

- changes are scoped and repeatable given the same inputs and policies
- upgrade decisions follow deterministic rules (version selection policy)
- verification outputs and reports are written to deterministic paths
- allowed nondeterminism (timestamps) is isolated from comparisons

---

6) Failure semantics

When MCP fails, it must still emit:

- `maintenance_run_report` with:
  - failing module/action
  - remediation steps
  - evidence pointers to logs and failing files
- partial outputs preserved (no overwrite)
- fail-fast on:
  - schema/contract breakage that blocks AXION compatibility (when required)
  - missing lockfiles or corrupted dependency state (if policy requires)
  - migrations that violate safety rules

---

7) Hard boundaries (locked)

- MCP does not redefine AXION system rules; it operates under them.
- MCP focuses on post-build lifecycle: evolution, hygiene, and long-term stability.
- MCP must respect scope constraints and may not expand changes beyond declared intent.
-

MCP Inputs

MCP Inputs (what it consumes)

1.1 Repo state (required)

Purpose: provide MCP with the complete, current technical reality of the repository it maintains.

- **Current repo source tree**
  - full working tree at a specific revision (commit SHA or equivalent)
  - includes configuration, scripts, and any generated artifacts that are checked in
- **Dependency manifests + lockfiles**
  - manifest files (language/package manager dependent)
  - lockfiles required for deterministic installs/builds
  - any policy files that constrain upgrades (pins, constraints, allow/deny lists)
- **CI workflow definitions**
  - pipeline configs and scripts
  - caching settings and toolchain setup steps
- **Test suite configuration**
  - test runner configs and scripts
  - test environment setup (fixtures, seed data, required services)
- **Migration history (if present)**
  - migration files + ordering metadata
  - migration runner configuration
  - schema history markers (if any)
- **Versioning metadata**
  - changelog and release notes (if present)
  - version tags and release identifiers
  - deprecation notes (if present)

Determinism requirements

- all “repo state” inputs must be anchored to a specific revision
- lockfiles must be treated as authoritative where required
- any generated files are either:
  - reproduced deterministically, or
  - excluded from MCP’s correctness checks by policy

1.2 AXION compatibility surfaces (required)

Purpose: define the contracts MCP must not break, even while evolving the repo.

- **AXION-required artifacts that the repo must remain compatible with** *(as applicable)*

    - kit artifacts (manifest/entrypoint/bundle metadata) if the repo produces or consumes kits
    - proof formats (proof objects, proof ledger, verification results) if the repo participates in proof capture
    - gate report formats if the repo runs verification/gating as part of CI or maintenance modes
- **Current MM-xx definitions or the repo's maintenance-mode mapping**
    - which maintenance modes exist
    - what each mode allows/skips/requires
    - never-skip requirements for maintenance work
- **Pin policies / toolchain constraints enforced by AXION**
    - required tool versions (node, package manager, compilers)
    - version pin rules and compatibility windows
    - required verification commands (doctor/verify/repro) if enforced

Determinism requirements

- compatibility surfaces must be loaded from a single authoritative location
- compatibility checks must reference schema/contract versions explicitly (no implicit assumptions)

1.3 Maintenance request / intent (required)

Purpose: constrain what MCP is allowed to do, so maintenance is controlled and auditable.

- **Maintenance mode id (MM-xx)** *or* **maintenance intent type**
    - examples: dependency upgrade, migration, refactor, test hardening, CI fix, security patch, rollback
- **Scope constraints**
    - allowed directories/modules
    - disallowed areas (protected paths)
    - change ceilings (e.g., "no API contract changes")
- **Risk class** *(prototype | production | compliance)*
    - determines strictness:
        - what can be skipped
        - what proofs are required
        - whether approvals are needed for certain changes

Determinism requirements

- intent type must map to a deterministic module execution plan
- scope constraints must be validated before changes begin
- risk class must deterministically select the verification profile

---

1.4 Optional inputs

Purpose: enable better incremental maintenance and faster triage without changing the required contract.

- **Prior maintenance reports and diffs**
  - used to detect repeat patterns and avoid rework
  - used for resume continuity
- **Prior failures**
  - CI logs, gate reports, proof artifacts
  - used to focus remediation and confirm fixes
- **Known issues / backlog references**
  - issue IDs or internal references that constrain/justify changes

Determinism requirements

- optional inputs may guide prioritization but must not override required policies/contracts
- if used, they must be referenced in the maintenance report for traceability

MCP State Model

MCP State Model (repo maintenance lifecycle)

2.1 Maintenance run states (minimum)

State sequence

- **PLANNED → APPLYING → VERIFYING → (BLOCKED | FAILED | COMPLETE)**

Optional

- **PAUSED**
- **CANCELLED**
- **ROLLBACKING**

State meaning (contract)

- **PLANNED:** maintenance intent is accepted; scope and policy constraints are known; no repo mutations performed yet
- **APPLYING:** repo changes are being applied (upgrades/migrations/refactors/test edits)
- **VERIFYING:** verification suite is running (tests/lint/build + compatibility checks where applicable)
- **BLOCKED:** work cannot proceed until a fixable condition is resolved (dependency conflict, failing tests, missing approvals)
- **FAILED:** non-recoverable failure under current constraints (policy violation, incompatible migration, irreconcilable dependency state)
- **COMPLETE:** maintenance goals achieved, verification passed, reports produced
- **PAUSED:** intentional pause with resumable state and preserved attempt history
- **CANCELLED:** terminated early; must preserve audit/report trail
- **ROLLBACKING:** rollback actions in progress to restore a prior known-good state

---

2.2 Work-unit states (minimum)

- **NOT_STARTED**
- **IN_PROGRESS**
- **DONE**
- **FAILED**
- **SKIPPED**

Work-unit meaning (contract)

- **NOT_STARTED:** unit exists but has not begun
- **IN_PROGRESS:** unit is actively being executed; partial changes may exist

- **DONE:** unit's intended change completed and required evidence exists (diff + verification pointers where required)
- **FAILED:** unit failed and cannot proceed without a new attempt or change of constraints
- **SKIPPED:** unit intentionally not executed per mode/policy with recorded reason

---

2.3 Transition rules (deterministic)

Run-level transition rules

- **PLANNED → APPLYING**
  - allowed only if:
    - **scope constraints are validated**
      - allowed paths confirmed
      - protected areas confirmed untouched by plan
    - **baseline health checks pass** *or* are explicitly waived by mode
      - baseline includes at minimum: dependency state readable, lockfiles present (if required), repo loads
- **APPLYING → VERIFYING**
  - allowed only if:
    - planned change set is applied (or partially applied with attempt record)
    - required intermediate artifacts (e.g., migration plan) exist when applicable
    - changes remain within validated scope constraints
- **VERIFYING → COMPLETE**
  - allowed only if:
    - required verification passes (tests/lint/build/typecheck as specified by mode/risk)
    - AXION compatibility checks pass (where applicable)
    - required maintenance report is produced and references evidence pointers
- **VERIFYING → BLOCKED**
  - allowed when:
    - verification fails in a recoverable way (policy allows iteration)
    - approvals are required before proceeding (e.g., major version bump under production risk)
    - dependency conflicts can be resolved without violating scope/policy
- **APPLYING/VERIFYING → FAILED**
  - allowed when:
    - policy violation occurs (scope breach, prohibited change)
    - verification failure is non-recoverable under current policy (e.g., breaking-change disallowed)
    - migration violates safety rules (e.g., destructive change without plan)
    - dependency state becomes invalid and cannot be restored within mode rules

- **APPLYING/VERIFYING → PAUSED**
  - operator action; must record who/why and preserve current attempt state
- **PAUSED → APPLYING** or **PAUSED → VERIFYING**
  - resume must:
    - preserve attempt history
    - re-validate scope constraints before continuing
- **Any → CANCELLED**
  - operator action; must write report with cancellation reason and current status
- **FAILED/VERIFYING → ROLLBACKING**
  - allowed only if rollback is enabled by mode/policy
  - rollback plan must identify a known-good reference (commit/tag/snapshot)
- **ROLLBACKING → COMPLETE** *(rollback complete)* or **ROLLBACKING → FAILED**
  - rollback must produce a rollback record and verification pointers showing restored health

---

Work-unit transition rules

- **NOT_STARTED → IN_PROGRESS**
  - allowed only if:
    - unit is within scope constraints
    - prerequisites are satisfied (e.g., migration depends on dependency bump)
- **IN_PROGRESS → DONE**
  - allowed only if:
    - unit outputs exist (diff applied, files updated)
    - required evidence exists (verification pointers if required for that unit)
    - changes remain within scope
- **IN_PROGRESS → FAILED**
  - allowed when:
    - unit violates policy constraints
    - changes cannot be completed under current mode/risk constraints
- **NOT_STARTED/IN_PROGRESS → SKIPPED**
  - allowed only if:
    - mode/profile explicitly allows skipping that unit type
  - skip must record a reason code and mode reference

Completion requirements (non-negotiable)

MCP cannot mark **COMPLETE** unless:

- verification passes as required by mode/risk
- AXION compatibility checks pass where applicable

- the maintenance report exists and links to evidence/log pointers

---

Rollback requirements (non-negotiable)

If rollback occurs:

- attempt history must be preserved (no overwrites)
- rollback record must be produced with:
    - what was rolled back
    - reference point restored
    - verification evidence after rollback

MCP Runtime Modules

MCP Runtime Modules (what it actually does)

3.1 Dependency Upgrade Manager

Function: bump dependencies safely and consistently.

Responsibilities

- apply version bumps according to policy (patch/minor/major rules)
- update lockfiles deterministically
- identify breaking changes (if detectable)
- record change notes

Produces

- `dependency_update_report`
- updated lockfiles
- changelog entry (if required)

Must enforce

- lockfile discipline (no drifting lock state)
- pinned toolchain requirements (if required)

3.2 Migration Manager

Function: manage schema/data evolution safely.

Responsibilities

- generate/apply migrations
- enforce migration ordering rules
- run backcompat checks (if required)
- create data repair scripts when needed

Produces

- `migration_plan`
- migration artifacts (files/scripts)
- `migration_verification_report`

Must enforce

- migration safety rules (no destructive changes without plan)
- rollback strategy recorded (if required by mode)

3.3 Test Suite Maintainer

Function: evolve tests to reduce drift and increase confidence.

Responsibilities

- add regression tests for known failures
- reduce flakiness (stabilize ordering/time assumptions)
- adjust coverage targets (if required by policy)

Produces

- `test_update_report`
- new/updated tests
- flake triage notes (if relevant)

Must enforce

- tests remain deterministic under pinned environment
- new tests map to failure causes (no “random” tests)

3.4 Refactor and Hygiene Manager

Function: restructure safely without breaking contracts.

Responsibilities

- apply controlled refactors (module boundaries, naming, folder moves)
- enforce lint/type fixes
- avoid breaking public interfaces unless change process followed

Produces

- `refactor_report`
- diff summary of moved/renamed components

Must enforce

- contract preservation (schemas, IDs, public interfaces)

- change-impact notes updated when required

3.5 CI Workflow Maintainer

Function: keep automation valid and aligned with system needs.

Responsibilities

- update workflows (steps, caching rules, tool versions)
- ensure `doctor`/`verify` remain enforced
- ensure artifacts are uploaded on failure

Produces

- `ci_update_report`
- updated CI files

Must enforce

- stable toolchain pins
- reproducibility checks preserved when enabled

3.6 AXION Integration Maintainer

Function: keep repo compatible with AXION contracts and ingestion.

Responsibilities

- validate that required AXION artifacts still match schemas
- validate that gate reports/proofs are still producible
- ensure kit outputs remain valid when this repo is a kit producer/consumer
- ensure compatibility when schemas/registries change (migration support)

Produces

- `axion_compatibility_report`

Must enforce

- schema compatibility rules
- version/pin compliance where required

3.7 Maintenance-mode Runner

Function: execute maintenance flows aligned to MM-xx modes.

Responsibilities

- interpret MM-xx for maintenance actions
- enforce mode-specific strictness and “never skip” rules
- enforce post-incident backfill (when mode requires)

Produces

- `maintenance_run_report`
- links to verification outputs and any override records

Must enforce

- mode rules are applied deterministically
- any override follows override policy and is recorded
-

MCP Output Artifacts

MCP Output Artifacts

O-00 Purpose

MCP outputs are the **repo-maintenance evidence spine**. They must make maintenance work:

- auditable (what changed and why)
- reproducible under pinned tooling
- safe relative to AXION compatibility constraints (when applicable)
- recoverable (rollback path recorded when used)

Always produced

O-01 `maintenance_run_report`

Role: the authoritative record of the maintenance run.

Must include

- maintenance run identity:
  - `maintenance_run_id`
  - repo revision baseline (commit/tag)
  - maintenance intent (MM-xx or intent type)
  - scope constraints (allowed/disallowed areas)
  - risk class
- execution summary:
  - modules executed (dependency/migration/test/refactor/ci/compat)
  - state transitions (PLANNED/APPLYING/VERIFYING/BLOCKED/FAILED/COMPLETE)
- change summary:
  - files touched summary (high-level)
  - dependency deltas summary (if applicable)
  - migration deltas summary (if applicable)
- verification summary:
  - commands executed + exit codes
  - log pointers
- compatibility summary (if applicable):
  - AXION checks run + results
- remediation block on failure:
  - what failed

    - why it failed
    - what to fix next
    - evidence pointers

Written when

- created at PLANNED
- updated at each major state transition
- finalized on COMPLETE/BLOCKED/FAILED/CANCELLED

O-02 Verification logs for executed commands

Role: raw evidence for tests/lint/build/typecheck outcomes.

Must include

- command id/name
- exit code
- stdout/stderr capture (or pointer)
- start/end timestamps
- environment/tool versions used

Pathing

- deterministic log paths per command and attempt

O-03 Status tracking for maintenance units (if unitized)

Role: machine-readable progress for unitized maintenance actions.

Must include

- unit list (if MCP breaks work into units)
- per-unit state:
    - NOT_STARTED/IN_PROGRESS/DONE/FAILED/SKIPPED
- attempt chain links (if reruns occur)
- pointers to produced reports/artifacts per unit

Requirement

- if MCP does not unitize work, this artifact may be omitted; the run report must still contain equivalent progress detail

---

Produced when relevant

O-04 `dependency_update_report`

Produced when

- dependency upgrades occur

Must include

- dependency changes (before → after) + version selection rationale
- lockfile updates summary
- breaking-change notes (if detected)
- rollback guidance (if required by mode/risk)
- verification pointers (tests/lint/build logs)

O-05 `migration_plan` + `migration_verification_report`

Produced when

- schema/data migrations occur

`migration_plan` **must include**

- ordered migration steps
- backcompat considerations (if required)
- risk notes and constraints
- rollback strategy reference (when required)

`migration_verification_report` **must include**

- migration execution results
- verification evidence (tests, schema checks)
- data repair actions taken (if any)
- pointers to migration artifacts

O-06 `test_update_report`

Produced when

- tests are added/modified, flakiness addressed, coverage targets updated

Must include

- what tests changed and why (regression linkage)
- flakiness fixes summary (if applicable)
- coverage changes summary (if applicable)
- verification pointers showing test suite passes

O-07 `refactor_report`

Produced when

- refactors or hygiene changes occur

Must include

- scope of refactor (modules/paths)
- interface/contract impact statement
- rename/move summary (diff mapping)
- verification pointers and any compatibility notes

O-08 `ci_update_report`

Produced when

- CI workflow changes occur

Must include

- what workflows changed and why
- toolchain pin changes (if any)
- caching rule changes (and nondeterminism risk notes)
- confirmation that doctor/verify checks remain enforced
- artifact upload behavior preserved

O-09 `axion_compatibility_report`

Produced when

- repo must remain compatible with AXION contracts (kit producer/consumer, proof formats, gate reports)

Must include

- which AXION surfaces were checked (schemas, manifests, proofs, gates)
- results and failures with evidence pointers
- required migrations or pin updates (if applicable)
- compatibility version references (schema/template/standard/gate versions)

O-10 Rollback record (if rollback executed)

Produced when

- rollback occurs

Must include

- what was rolled back (commit/tag/snapshot reference)
- rollback reason
- rollback steps performed
- verification evidence after rollback
- preserved attempt chain links (no overwrite of prior reports)

MCP Determinism guarantees

Determinism guarantees (repo maintenance)

D-00 Purpose

MCP determinism ensures maintenance actions are **repeatable and auditable**: the same repo revision, the same maintenance intent, and the same policies produce the same decisions and comparable outputs.

D-01 Deterministic “input set” definition

MCP evaluates determinism against a canonical set:

- repo baseline revision (commit/tag)
- maintenance intent:
  - MM-xx or intent type
  - scope constraints
  - risk class
- upgrade/migration policies (version selection rules, safety rules)
- toolchain pins (node/package manager/compiler versions)
- verification policy (which checks must run)

Optional inputs (prior failures/backlog) may guide prioritization but must not override policy decisions unless explicitly permitted.

D-02 Scoped and repeatable changes

For the same inputs and policies:

- MCP must generate a change plan that stays within the declared scope constraints
- MCP must refuse any change that expands scope without an explicit scope update
- MCP must record scope validation results before applying changes

Repeatability requirement

- if rerun from the same baseline revision with same policies, the intended change set must be identical (modulo allowed nondeterminism fields in reports)

D-03 Deterministic upgrade decisions

Upgrade outcomes must follow a deterministic version selection policy:

Requirements

- no implicit “latest” unless policy explicitly allows it
- if ranges are allowed, tie-breaking rules are fixed (e.g., highest patch within allowed minor)
- policy must define:
  - allowed update classes (patch/minor/major)
  - allow/deny lists for packages
  - constraints for breaking changes under risk class

Recorded evidence

- `dependency_update_report` must record:
  - selection policy used
  - final resolved versions
  - lockfile delta pointers

D-04 Deterministic migration decisions

Migration ordering and selection must be deterministic:

Requirements

- ordering rules are fixed (timestamp/sequence ids)
- no reordering without explicit migration metadata change
- safety checks are deterministic (same migration file → same safety classification)

Recorded evidence

- `migration_plan` includes ordered list and safety classification decisions

---

D-05 Deterministic verification outputs and report pathing

Verification logs and MCP reports must be written to deterministic paths derived from stable IDs:

Path inputs

- maintenance_run_id (allowed noise, used only as run scope)
- module name (dependency/migration/test/refactor/ci/compat)
- command id/name

- attempt id for reruns

Requirements

- no ad-hoc filenames
- reruns produce new attempt-scoped outputs, never overwrite
- reports and logs are discoverable via the maintenance_run_report pointers

---

D-06 Allowed nondeterminism isolation

Allowed nondeterminism must be confined to explicit fields:

- timestamps
- generated run ids
- environment fingerprints (runner id, hostname)

Rules

- nondeterministic fields must not affect upgrade selection or migration ordering
- comparisons and goldens must normalize these fields out before diffing

---

D-07 Determinism evidence

MCP must emit enough evidence to validate determinism:

- baseline revision recorded
- policy identifiers recorded (which upgrade/migration policy was applied)
- toolchain pins recorded
- resolved dependency versions recorded (when upgrades occur)
- report pointers and hashes recorded for key outputs (when enabled)

MCP Failure semantics

Failure semantics (MCP)

F-00 Purpose

An MCP failure is a **controlled stop** that preserves the repo's audit trail and makes the failure diagnosable. Failure must never result in silent drift, overwritten evidence, or ambiguous "what happened" states.

F-01 Required outputs on failure (always)

When MCP enters `BLOCKED` or `FAILED`, it must still emit:

F-01.1 `maintenance_run_report` (failure-complete)

Must include:

- final maintenance run state (`BLOCKED` or `FAILED`)
- failing location:
    - **module** (dependency upgrade / migration / tests / refactor / CI / AXION compat / mode runner)
    - **action** (the specific step that failed)
- failure reason code(s)
- remediation steps (actionable, sequenced)
- evidence pointers to:
    - verification logs
    - failing files/diffs
    - any compatibility check outputs (if applicable)
- scope and policy context:

    - scope constraints at time of failure
    - risk class
    - mode/intent id

F-01.2 Evidence pointers (logs + file refs)

If any verification commands ran:

- command id
- exit code
- log pointer(s)
  If file-level failures occurred:
- file path pointers and/or diff pointers to the failing changes

F-01.3 Partial outputs preserved (no overwrite)

MCP must preserve:

- any module reports already produced (dependency_update_report, migration_plan, etc.)
- partial diffs (if changes were applied)
- attempt history (if retries occurred)
  No partial output may be overwritten; subsequent attempts must be versioned or attempt-scoped.

F-02 Fail-fast conditions (hard stops)

MCP must fail-fast immediately when any of the following occur:

F-02.1 Schema/contract breakage that blocks AXION compatibility (when required)

Examples:

- AXION-required artifacts no longer validate against schemas
- kit manifests/proof formats/gate reports become invalid and cannot be migrated under policy
- version/pin compliance rules are violated with no allowed override path

Disposition

- transition directly to `FAILED` unless policy explicitly allows a rollback attempt

---

F-02.2 Missing lockfiles or corrupted dependency state (if policy requires)

Examples:

- lockfile missing when lockfile discipline is mandatory
- lockfile cannot be parsed
- dependency state cannot be resolved deterministically (install cannot stabilize)

Disposition

- transition to `BLOCKED` if fixable (regenerate lockfile within scope)
- transition to `FAILED` if policy forbids regeneration or state cannot be stabilized

---

F-02.3 Migrations that violate safety rules

Examples:

- destructive changes without an approved migration plan
- unsafe ordering or irreconcilable migration conflicts
- irreversible data changes without rollback strategy when required

Disposition

- transition to `FAILED` (migration safety violations are non-negotiable unless explicitly overridable by mode)

F-03 BLOCKED vs FAILED classification (deterministic)

MCP must classify failures as:

BLOCKED (recoverable under same intent)

- failing tests that can be fixed within scope
- dependency conflicts that can be resolved within policy
- missing approvals required by mode/risk class

FAILED (non-recoverable under current constraints)

- contract/policy violations (scope breach, non-overridable compat break)
- unsafe migrations
- dependency state irreparably corrupted under policy
- repeated verification failures beyond allowed retries (if defined)

Classification must be recorded as reason codes in `maintenance_run_report`.

F-04 Attempt handling on failure

If MCP retries a module/action:

- each retry must create a new attempt record
- prior attempt logs/reports remain immutable and referenced

- the run report must link attempts in order and identify which attempt produced the final failure

---

F-05 Minimal failure correctness criteria

An MCP failure is “well-formed” only if:

- failing module/action is identified
- remediation steps are present
- evidence pointers resolve
- partial outputs and attempts are preserved and referenced

MCP Hard boundaries

Hard boundaries (locked) — MCP

B-00 Purpose

MCP boundaries ensure maintenance work stays **non-authoritative**, **scope-controlled**, and **compatible** with AXION contracts. MCP maintains; it does not redefine.

B-01 MCP does not redefine AXION system rules

Meaning

- MCP treats AXION rules (standards, templates, gates, schema contracts, pin policies) as authoritative constraints.

Allowed

- running compatibility checks against AXION-defined schemas/contracts
- updating repo artifacts to remain compatible *when policy allows* (e.g., migrations aligned to published schema changes)
- producing maintenance reports and evidence that demonstrate compliance

Not allowed

- changing AXION rule definitions (standards/templates/gates) as part of repo maintenance
- weakening enforcement expectations ("skip doctor/verify") unless an explicit maintenance mode authorizes it and it is recorded
- inventing new compatibility interpretations that contradict AXION contracts

Boundary enforcement expectation

- MCP must reference the current AXION contract versions/pins when performing compatibility checks
- any exception must be recorded as an override per override policy

---

B-02 MCP focuses on post-build lifecycle: evolution, hygiene, long-term stability

Meaning

- MCP operates on maintenance surfaces, not feature delivery.

Allowed

- dependency upgrades, migrations, refactors, test/CI hardening, security patches
- performance/cost hygiene (within maintenance scope)
- operational readiness improvements (observability config, alerting hygiene)

Not allowed

- adding net-new product features unrelated to the maintenance intent
- changing product requirements/spec goals (that belongs upstream in AXION planning/spec)
- expanding maintenance into “feature work” without a new, declared intent

Boundary enforcement expectation

- maintenance intent must be stated and reflected in the maintenance report
- changes must be attributable to that intent (traceable change rationale)

B-03 MCP must respect scope constraints; may not expand changes beyond declared intent

Meaning

- MCP’s authority is bounded by explicit scope constraints (paths/modules/change ceilings).

Allowed

- modifying only whitelisted areas
- making necessary collateral changes that are explicitly permitted by policy (e.g., lockfile updates with dependency bumps)

Not allowed

- touching protected paths
- refactoring unrelated modules “because it’s cleaner”
- broad dependency churn outside allowed policies (e.g., major bumps in a patch-only mode)

Boundary enforcement expectation

- preflight scope validation required before APPLYING

- scope violations must fail-fast and be recorded with file pointers
- any scope expansion requires an explicit scope update and a new record in the maintenance run report